

Next：以直接担保实现连续 UTXO 支付

Lu Hengrun

摘要—连续支付允许接收方在创建其输出的交易公开执行之前花费该输出。Next 是一个原生 UTXO 账本：接收方在一轮认证后验证由法定人数签名的输出证书 (TXCer)，并可在公开处理继续进行的同时请求后继交易。若后继交易先执行，委员会会针对直接发行方记录一项有资金支持的义务。原签名者可以根据来源交易公开暴露的证书重新提交该来源交易。由截止期触发的决定会扣减发行方的备付金，迟到的来源交易则将其偿还。随后，可选的授权历史命令在原始坐标处记录备付引用，同时保持所有者授权、后继引用以及完整 BlockID 不变。对于由四名成员组成的组织和由四个验证者组成的委员会，二者均要求三票且至多含一个静态拜占庭成员，我们证明了唯一消费、未偿证书责任的覆盖，以及对来源已闭合且无冲突的支付集合而言延迟中性的最终 CAL 持有量。在成员同步提交的单台主机上，100 跳链到达其最后一次已认证接收的中位时间为 4.557s，而逐跳等待时为 64.476s。针对性的边界与重启测试验证了关于额度、经济闭合和历史读取的相关条件。

关键词—区块链，认证支付，抵押，拜占庭容错，变色龙哈希。

I. 引言

在 UTXO 账本中，每笔支付都花费先前交易的输出，因此若接收方等待公开执行，支付链的每一跳都会增加一次确认延迟。假设 Alice 向 Bob 支付 100 CAL，而 Bob 在 Alice 的支付执行之前将其转给 Carol。若 Bob 的支付先执行，账本将消费一个尚不存在的输出，缺失的 100 CAL 就需要一个明确且有资金支持的来源，无论 Alice 的交易稍后到达还是永不到达。

在每一跳都等待公开执行是最直接的替代方案，我们的评估将其用作对照。基于证书的系统把接收方的可用性与公开处理分离开来：FastPay 和 Zef 支持低延迟的认证支付，Sui Lutriss 则针对不同类型的状态访问结合了广播路径与共识路径 [1]–[3]。支付通道依靠通道流动性实现快速转账 [4], [5]。Next 要问的是：原生账本如何为来源尚未执行的认证支付提供资金，其背后由已注册的组织而非通道作为支撑。

三个困难相互耦合。证书在公开登记之前就已流通，因此仅依据公开缺口进行核算会遗漏已存在的承

诺。迟到的来源交易不得既向垫付价值的担保方付款，又向已经花掉该价值的接收方付款。客户端可能持有某个来源交易却永不提交。

我们的核心观察是：对缺失来源负责的一方，已经由随输出流转的证书指明。Next 将缺口记在该直接发行方名下，而不是追溯祖先交易；以由备付金支持的签名额度约束每份已签证书的责任；并通过公开决定关闭未解决的义务，而不改变已存储的历史。该设计包含三项贡献，核心是对缺失来源的直接责任。

C1：支持一轮继续的分层架构。认证与公开执行承担不同的职责：证书固定输出并将责任归于其发行方，而公开执行决定账本接受哪笔支付。消费组织的成员在签名前原子地锁定输入并预留额度，一轮并行交互即产生一个 TXCer。接收方验证并记录该输出后可立即请求后继交易，且只携带其直接输入的证书，复制与公开提交则在后台运行。

C2：带来源解耦的直接责任。公开执行以原子方式消费缺失的已认证输入，并针对其发行方记录一项义务。后继交易所在区块公布来源交易的证书，因此原签名者可以根据存储的批准重新提交该来源交易；截止期之后，公开决定无需门限适配即可扣减发行方的备付金，迟到的来源交易则将其偿还。每个批准在仍可能被认证期间预留完整的 CAL 输出（含找零），而经认证的冲突性公开花费可以释放未获证批准的未使用本地预留。对于由四名成员组成的组织和由四个验证者组成的委员会（各自至多含一个静态拜占庭成员），我们证明了唯一消费，以及签名额度覆盖每份证书（无论是否已公布）未偿 CAL 责任这一性质。对于一个有限、无环、无冲突、包含其已认证输入之来源交易且每笔支付只执行一次的支付集合，来源到达与赔付的所有顺序最终得到相同的 CAL 持有量和备付余额。该核算将成员间重叠的预算 [6] 扩展到隐藏的证书责任、动态赔付与偿还以及来源闭合。

C3：可选的授权历史表示。受限变色龙哈希命令将已赔付输入的资金引用替换为备付引用，同时保持所有者授

权、后继引用以及完整 BlockID 不变。已执行前缀 H 的诚实副本随后可在原始坐标 (h, j, k) 处提供原始与授权后的区块体字节、完整 BlockID、授权的 Revision 与 Task、历史资金引用以及当前义务状态。永久决定仍是经济依据，原始命令的重放保持不变。该层不在支付路径上，并带来额外的记录与本地维护开销；一个可运行的本地消费者演示了字节导出契约。

在一台采用成员同步提交与非同步钱包存储的 M4 Max 主机上，100 跳链到达其最后一次已认证接收的中位时间为 4.557 s，而同一系统在每一跳都等待公开执行时为 64.476 s。一个由组织支付 FUEL 的十四服务混合负载在有界并发下完成 84,192 笔支付；在重新提交处理组中，原签名者解决了全部 24 个被扣留的来源交易，并且在所有适配服务暂停期间赔付与偿还均完成。在同一主机上的十二轮研究中，对成员 RPC 与委员会 P2P 流量每方向增加 25 ms，使已认证接收 P50 从 41.6 ms 升至 81.9 ms；暂停一个成员或一个委员会验证者 30 s，已认证接收 P95 保持在 109–113 ms；全部 15,840 笔支付均闭合。在无对象缓存的本地热页读取中，对所测布局，与优化后的决定索引相比，表示使点读 P50 降低 7.44–12.32%，每个被修复的区块增加 0.28–2.19 MiB 并增加一个 43–214 ms 的维护步骤。这些读取测量刻画了该配置下可选层的代价。

II. 背景与相关工作

Next 结合了三类要素：接收方可在公开执行之前复用的认证支付、对尚未执行的来源由备付金支持的责任，以及对已存储历史的授权编辑。我们沿两个问题与已有工作比较：谁对缺失的来源负责，以及在该责任被记录之后，哪些支付与区块身份得以保留。

基于证书的支付与执行。 FastPay 使用拜占庭一致广播实现低延迟支付，并在其主账本部署中以预存资金支持赎回 [1]。其可赎回性论证允许在所有入账转账尚未在本地确认之前就确认一份有效证书，条件是提供发送方缺失的较早的出账证书；赎回随后到达主资金池。Zef 将一次性账户与私密、可转让的币相结合，普通花费使用当前币证书和消费证据，而不是完整的祖先历史 [2]。Sui Lutriss 对自有对象采用广播执行，对共享对象交互采用共识 [3]。

Stingray 通过拜占庭有界计数器与 FastUnlock 支持并发交易 [6]。其重叠的逐副本预算（第 4.2 节）以及对不形成法定人数证书时预算耗尽的处理（第 5.3 节）是

Next 签名额度核算的计数先例，我们不将其表述为新贡献。其可交换的工作负载不同于支付链，后者中每笔交易都消费其前驱的输出。针对这类链，Next 在来源交易执行之前公开执行已认证的消费，并针对直接发行方登记一项有资金支持的义务。Next 增加了对从未公布的证书的责任、每份证书的 Paid、Recovered 与 Remaining 量的动态守恒，以及通过公开决定关闭来源义务。后继交易的执行同时公布来源交易的证书，因此发行组织自己的成员可以根据其存储的批准重新提交该来源交易。

通道与有抵押的担保。 Blitz 以 pay-or-revoke 取代通道支付中的两阶段承诺 [4]，Donner 发展了应对 Domino 攻击的虚拟通道 [5]，CryptoMaze 提供带不可链接部分支付的原子多路径通道支付 [7]。这三者都通过通道安排运作。Next 运行在带有已注册消费路由和组织备付金的原生账本上，因此交易对手既不需要双边通道，也不需要二者之间预先注资的路由。

Snappy 通过有抵押的批准保护接受快速链上支付的商户，可复用的支付担保则将隐私与可在既定时间窗口内复用的客户抵押相结合 [8], [9]。Next 将用户的支付本金与额外的组织支持分开，其赔付关闭的是已执行子交易所缺失的来源，而不向该子交易的接收方再次计入本金。HyperService 通过公共的 Network Status Blockchain 和 Insurance Smart Contract 裁决跨链执行违规，并在经济上逆转已提交交易的影响 [10]。Next 在单一原生 UTXO 账本之内划定责任边界：赔付结算的是直接发行方义务，并保留已执行后继交易的本金效果。

受控编辑与历史一致性。 变色龙哈希允许在同一摘要下使用经授权的替代消息，并构成可编辑区块链设计的基础 [11]。SDR 研究去中心化的基于属性的编辑权限与动态委托 [12]，Escaping 将委员会选择与投票移出共识层以缩短删改等待时间 [13]，Reconstructing Chameleon Hash 则为不可区分性与抗碰撞性提出了更强的定义 [14]。Next 使用成熟的适配原语，其陷门以 RSA 指数的门限份额形式持有，遵循 Shoup 的实用门限签名方法 [15]。应用兼容替换的权限来自已提交的赔付决定与公开的支付状态检查，而不是来自该原语。

Reparo 提供可公开验证的修复层，并可在修复状态时重新执行受影响的交易 [16]。Next 预先固定支付授权与输出：公开决定只计入一次赔付，表示命令仅替换被允许的资金表示，而不重新执行后继交易。原始命令（包括这些决定）仍是经济重放历史。RedactChain 协调对同一区块的并发编辑及其预处理 [17]，CDEdit 使

用可复用的权限令牌，以削减多次编辑中重复的令牌与密钥生成及验证 [18]。在 Next 中，一个批次在同一区块中各已决定槽位之间共享最终区块表示，且每个槽位的备付扣减已由其自身的决定提交，因此该编辑不产生经济效果。完整区块身份将稳定的区块头哈希与原始 part-set 头结合，公开提交的逻辑修订则授权精确的替换区块体。物理安装因此可以直接推进到较晚的授权修订，而不改变后继引用或公开命令历史的重放。

III. 系统模型与概述

Next 让接收方在创建某个新认证输出的交易公开执行之前就能花费该输出。设计由四类对象承载：经认证的支付权利、其公开消费、对未解决来源的责任，以及该责任之后在已存储历史中的表示。表 I 列出了符号。

A. 参与者、资产与假设

Next 运行一个原生 UTXO 账本。CAL 承载支付价值，FUEL 支付处理费用。用户花费自己的输出，组织在公共备付账户中持有额外的 CAL。快速路径要求输出带有受支持的组织路由；支付本金由其所有者持有，且无需开设双边通道。费用默认由用户的最终 FUEL 输出支付；同时也存在经授权的由组织支付的模式。

每个组织有四名固定成员和一个可替换的收集者。成员独立验证请求并授权支付，收集者则分发请求、汇集证书并协调传播，不具有花费权限。一个独立的、由四个权重相等的验证者组成的委员会 C （需三票）对公开命令排序，并维护账户、输出、义务与赔付决定。

收集者访问与部署条件。任何客户端都可以转发经所有者授权的请求；担任收集者既不授予花费权限，也不授予由组织支付费用的权限。成员仍然强制执行所有权、输入锁、资源 grant 以及适用的费用策略。这些逐请求

的守卫并不强制跨收集者的未完成批准的系统级上界，而网关的并发限制也不是这样的上界。因此，第 V-F 节中的额度条件描述的是受控工作负载或外部部署限制。若没有该限制，组织需接受其中所分析的额度滞留与备付损失边界。

故障与密码学。对手在整个执行期间静态地至多腐蚀每个组织的一名成员和一名委员会验证者；用户与收集者的行为任意。诚实参与者共享网络配置、创世、规则与组织注册表，每条路由指定一个活动配置和一个消费锁域。签名不可伪造，规范摘要与原始区块承诺具有绑定性，消息可被任意延迟。

状态。诚实的签名状态是原子的，且从不被遗忘或回滚。成员签名之前，其批准记录了该请求的支付、准入向量和直接输入证书；法定人数证书不属于其中。公开执行是确定性的并以原子方式提交，跟随者应用经认证的连续区块，同时更新状态与游标。赔付与表示读取原始区块及其元数据，因此正确的委员会副本在每个存在未结义务或待表示的已决定槽位的高度上都保留这两者。

适配与进展。表示使用门限适配，其材料由可信的离线 dealer 生成并分发，且任何适配输出都必须由已提交的命令授权。进展需要响应的诚实参与者、可用的证据、充足的费用与资源，以及公平的交付与调度。实验用的内存配置仅在进程运行期间保持状态原子，且每次运行都从全新的创世开始。

历史读取涉及一个已验证并执行公开前缀 H 的诚实副本，在其状态 V_H 上使用一个读事务。第 IV-H 节规定了接口语义。

B. 输出、路由与证书

输出携带一种资产、一个金额、一个所有者和一条消费路由。其标识符 x 由网络、创建交易和输出位置导出，花费以 (x, i) 索引，普通输出的 $i = 0$ 。快速支付的所有输入共享同一个消费组织，并由所有者授权该交易。新输出可以选择另一个已注册的组织，因此跨组织的继续只改变新输出的路由，而之前发行方的责任保持不变。

TXCer 将摘要与法定人数证书 (QC) 配对：QC 是对同一事实 c_T 的三个不同成员签名。摘要绑定网络、交易身份、发行方、配置、纪元、规则、输出承诺与金额以及准入 grant。输出体可以单独交付，接收方在保存输出之前检查承诺、位置与所有权。该证书认证的是发行方的担保，它既不是资产，也不是已登记的公开义务，更不是对公开执行或偿还的无条件担保。

表 I
主要符号。

符号	含义
G, C	担保组织与公共委员会
T, c_T	支付及其经认证的摘要事实
(x, i)	输出标识符与花费实例
g, B_g	Grant 与累计授权额
a_T, p_T, Rec_T	CAL 输出总额、赔付总额（毛额）与已偿还的赔付额
$\mathcal{O}_x, \mathcal{D}_x$	针对缺失输出的直接义务及其不可变的赔付决定
(H_b, P_b)	稳定区块哈希与原始 part-set 头
ρ_ℓ, ρ_p	已提交的逻辑修订与已安装的物理修订

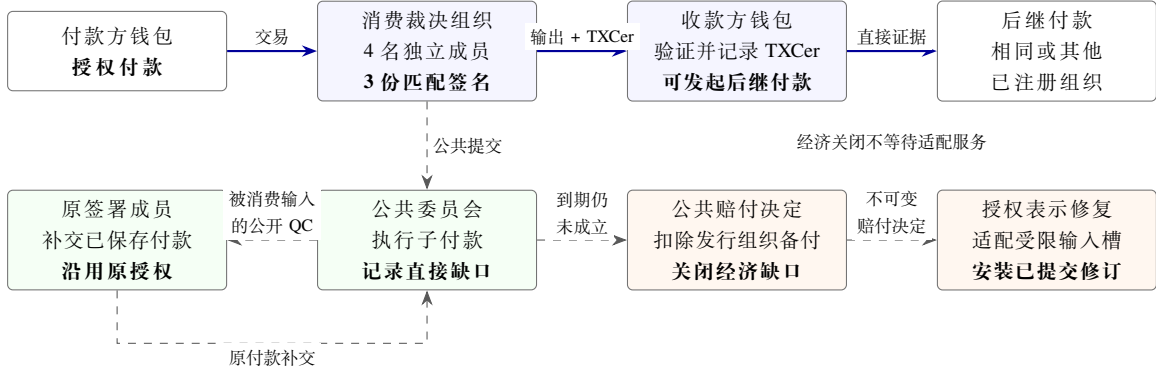


图 1. 单轮续花与公开证据驱动的来源处理。实线表示收款人路径，虚线表示后台工作。后继公开证据使原签署成员能够补交已保存的授权材料。来源到期仍未成立时，委员会独立完成赔付，不依赖适配服务。后续授权表示修复保持付款身份和完整区块身份。

每笔支付携带

$$\text{IntentID} = \text{Digest}(\text{INTENT}, \text{Network}, \text{Subject}, \text{Nonce}),$$

它命名的是一个付款方在一个网络上的一次操作；它并非在用户之间共享的 `nonce` 空间。成员按组织保存本地 `intent` 记录，而公开执行则在成功执行该支付的同一原子转换中，将一个 `intent` 绑定到唯一的 `TxID`。因此，两个组织可以基于相互独立的输入，认证两笔具有相同 `subject` 与 `nonce` 的支付并都获得 `QC`，但至多一笔在公开层执行。

三类事件反复出现：已认证接收，即钱包在本地验证并记录一个输出；公开执行，即已提交的命令消费输入并创建输出；以及责任闭合，即缺失的来源被履行或被赔付。赔付在经济上关闭责任，表示则随后在已存储历史中记录该闭合。接收为后继请求提供证据，而后继请求要经过其自身的授权与资源检查。在全文中，输出的来源（亦称其父）是创建它的支付，其消费方（亦称其后继或子）是花费它的支付。子输出是消费方所创建的输出。来源到达指来源交易的成功公开执行，实现中的 `Settled` 标志标记这一状态，且它可以与非零的 `Pending` 并存。

C. 三层设计

认证与继续。成员在签名之前原子地预留输入实例和适用的资源。CAL `grant g` 授权 B_g ，每个成员持有 $\lfloor 2B_g/3 \rfloor$ 的签名额数，并在其本地执行 `Worker` 之间划分。一笔支付预留 a_T ，即其所有 CAL 输出（含找零）之和；这些重叠的额度约束承诺，但不增加支持资金。收集者在一轮并行的请求-响应中获得三个匹配的批准后返回 `TXCer`。

接收方验证与原子的本地记录完成快速路径，而中继材料存储、`INSTALL`（组织内完整材料的复制）以及公开提交则在后台运行。输入锁的存续时间长于预算释放。

来源解耦的公开执行。公开命令携带交易、准入向量、组织授权和直接输入证书。委员会重建经认证的摘要，验证授权，并检查当前消费状态。对于尚未公开创建的已认证输入，它以原子方式记录该消费、针对发行方登记整份证书范围的覆盖、创建 $\mathcal{O}_x$ 及其缺口并创建子输出；只有实际被消费的缺失输出才会产生义务。每次公开支付检查都使用直接证书，而不是获取祖先交易体。成员根据经认证的区块，对照自己所记录的扣减恢复签名额数（第 IV-C 节）。创世时按每个支持账户检查汇总的 `grant`，充值从未受保护的账户扣款，并以原子方式提高余额与 `grant`；授权额度永不减少。

重新提交、赔付与表示。义务通过来源到达而关闭，或在由公开区块时间确定的截止期之后通过发行方备付金的赔付而关闭。执行消费方的区块公布其来源的证书，持有该批准的发行组织成员根据保留的材料重建支付并在无需新签名的情况下重新提交。截止期过后，公开赔付决定可以关闭仍处于 `Open` 状态的义务。它扣减备付金、关闭缺口，并记录一个不含适配数据的不可变决定；之后到达的来源交易在其自身的执行中向赔付的发行方偿还，而该决定仍保持记录。

表示命令随后将已决定输入的资金引用替换为备付引用。受限变色龙哈希适配保持固定的授权、输出引用以及完整区块身份 (H_b, P_b) ；`RepairBatch` 对同一历史区块中已决定的槽位分组，且这些命令只写入表示状态。原始命令、已提交的修订 ρ_ℓ 与已安装的修订 ρ_p 保持分离，安装可直接推进到较晚的授权版本，重放则运行包含这些决定在内的原始命令。后继交易既不需要新

签名，也不需要重新执行。

D. 连续支付示例与设计目标

Alice 通过 G_A 向 Bob 支付 100 CAL，创建路由到 G_B 的输出 x 。Bob 验证其 TXCer 并获得一份向 Carol 支付的证书。若 Bob 的支付先执行，委员会消费 $(x, 0)$ ，针对 G_A 记录一项 100 CAL 的义务，并创建 Bob 的输出。Carol 无需 Alice 的交易体即可从 Bob 的支付继续，且携带 Bob 支付的区块同时携带 Alice 支付的证书，因此批准过它的 G_A 成员可以在 Alice 的钱包不重新提交时代为重新提交。

随后有三种结果。若 Alice 的支付在赔付之前执行，它关闭该义务，实例 0 保持已消费状态。截止期过后，赔付决定可以通过从 G_A 扣减 100 CAL 来关闭仍处于 Open 状态的义务；Carol 不会再获得一次本金计入，Bob 的支付不会重新执行，已存储的区块保留原始资金引用，直到某条表示命令将其替换为备付引用。若 Alice 的支付在赔付之后执行，它按相同规则消费自己的输入，并向 G_A 偿还 100 CAL；该决定仍保持记录，Bob 不会获得第二个输出， $(x, 0)$ 保持已消费状态。若 Alice 的支付从未执行，则由 G_A 承担损失。

IV. 协议设计

Next 认证固定的支付授权与输出，并允许它们的来源保持未解决状态。公开执行以原子方式消费缺失的已认证输入并记录发行方的直接义务，同时公布来源交易的证书，使原签名者能够重新提交该来源交易。截止期时仍缺失的来源经由无需门限适配的公开赔付决定而关闭，表示命令随后在既有承诺之下替换资金引用。

A. 身份、认证与交付

令 K_T 为编码后的支付体与输入声明， C_T 为输入承诺， Σ_T 为输出摘要：

$$\begin{aligned} K_T &= \text{Encode}(T.\text{Body}, T.\text{Claims}), \\ \text{id}(T) &= H(\text{TX_V3}, K_T, C_T), \\ c_T &= H(\text{DIRECT_CERT_V3}, \text{Encode}(\Sigma_T)). \end{aligned} \quad (1)$$

所有者签名认证 $\text{id}(T)$ 且不参与其计算，每个输出标识符绑定该身份、网络和输出位置。资金引用与开口单独编码，因此受限替换不改变已授权的声明与输出。初始提交要求每个被声明的输入具有 OriginalFunding 及

有效开口，而后续表示保持支付体、声明、承诺和所有者签名不变。因此验证缓存以完整命令字节为键。

在同一 Network 与 Subject 之下，诚实钱包对每笔新支付（跨组织与实例）使用新的支付 nonce；重试则以其原授权重新提交同一笔支付。钱包不得更改 nonce 而复用旧证书，共享同一 intent 的两个事实也不会被合并为一项担保。实验驱动程序从输入标识符导出 nonce，这并不是通用的多设备 nonce 分配服务。

成员检查所有权、路由、直接输入证书、价值守恒和准入向量。一个本地事务重新检查输入锁、最终费用输入以及指定 Worker 的资源切片，并连同其精确的扣减一起记录批准；该批准保留请求的交易、准入向量和直接输入证书（第 IV-D 节）。即使每个本金输入都是最终的，CAL 预留也等于包含找零的 a_T ，因为每个已认证输出都可能在支付执行之前流转。成员在该事务提交之后签名，重试复用未被作废的批准，而释放资源额度则保留消费锁。

收集者向四名成员发送一个规范请求，在验证了针对同一 c_T 的三个签名后返回；不同的有效子集给出同一事实。接收方对照输出体、索引、所有者和可信配置检查 TXCer，原子地记录该币，并可立即构造后继请求，其中携带其直接输入证书以代替祖先链。

中继材料在后台保存，有界的早期任务使 INSTALL 与公开提交能够相互独立地进行。INSTALL 在成员处存储完整材料并在本地记录已认证的消费；它既不创建新的批准，也不产生预算扣减。公开提交单独进行。成员相对于其第一次 INSTALL 来安排备份提交，重复的 INSTALL 不能推迟它。HTTP 接受使发件箱条目保持待处理状态，直到经认证的公开完成将其移除。备份提交需要保留的完整提交；第 IV-D 节增加了一条基于原签名者批准的路径。

公开的 DirectSubmission 携带交易、准入向量、组织 SpendQC 和直接输入证书。委员会重建 Σ_T 并验证同样的授权；省略单独的新输出 TXCer 封装只改变信封而保持该要求。公开验证将每份输入证书绑定到其网络、发行组织、事实和被声明的输出，并拒绝重复。它对每个证书输入恰好接受一份证书，并拒绝任何多余的证书。准入向量含一个 CAL 条目，且绑定到发行方。

B. 带直接义务的原子执行

公开执行将整份证书范围的覆盖与实际的缺失输入义务区分开。在父事实 c 第一次被缺失使用时，

register 预留其全部已认证的 CAL 金额并为每个输出创建一个 Promise。覆盖记录维持

$$\text{Original}_c = \text{Discharged}_c + \text{Paid}_c + \text{Remaining}_c, \quad (2)$$

其中 Paid_c 是毛额累计计数器, $\text{Rec}_c \leq \text{Paid}_c$ 统计由来源执行偿还的部分。Grant 用量满足 $\text{Reserved}_g + \text{Spent}_g \leq B_g$, 其中 Spent_g 对 g 之下各证书的 $\text{Paid}_c - \text{Rec}_c$ 求和。登记提高公开的 grant 用量而不触动备付账户; 一次生效的赔付 (第 IV-E 节) 扣减该账户并提高 Spent_g , 偿还则贷记该账户并将 Spent_g 降低同样的数额。只有实际被消费的缺失输出才会产生直接义务并提高公开缺口, 因此消费一份 40/60 证书中缺失的一个 40-CAL 输出, 会预留 100 CAL、产生一个 40-CAL 的缺口, 并使未使用的 Promise 保持被覆盖。

图 2 给出公开转换。其前置状态包含同一区块中较早的成功命令; 静态授权可被缓存, 而消费、余额、grant 和义务则对照当前状态评估。所有变更都发生在私有 overlay 中, 失败将丢弃整个转换。

intent 检查属于此评估: 若 T 的 intent 已被绑定到不同的 TxID, 则该转换返回冲突。该绑定在成功执行一笔支付的同一原子转换中写入, 且没有任何生产路径删除或重新绑定它, 因此重新提交相同字节不能消除该冲突。

义务绑定被消费的输出、父证书事实、发行方、金额、消费交易以及历史输入位置, 其截止期以公开区块时间为锚。消费支付记录其 Pending 输入义务与费用进度。用户支付的费用消费经认证的最终 FUEL, 将多余的输入价值作为找零返还, 并托管已签名的最大值, 该最大值涵盖登记、结算、闭合以及每个已认证输入的一次赔付费用; 唯一的阶段与义务标识符防止重复收费, 未使用的托管在输入义务闭合时返还给付款方。即使部分输入仍是义务, 子输出也是最终的, 因此后续交易可依据普通的最终输入检查消费它们。

C. 来源闭合、偿还与累计对账

来源交易通过普通的支付执行到达, 包括验证并消费其自身的输入, 其输出根据各自的义务在该转换内被处理。到达将 Open 义务关闭为 Fulfilled: 它履行 Promise、解除覆盖并降低消费方的 Pending 计数。没有任何支付消费的输出会履行任何已登记的 Promise 并成为最终的实例 0。Repaired 义务意味着由备付扣减为消费方提供了资金, 因此到达时偿还该扣减, 而不是创建

Require: 已验证的提交 T , 当前状态 S , 公开时间

- 1: 若 T 已 settled, 则返回无新效果
- 2: $O \leftarrow \text{Overlay}(S)$
- 3: 检查 intent、grant 和 work; 在 O 中为费用托管注资
- 4: **for** 每个 CAL 输入 (x, i) **do**
- 5: 要求 (x, i) 未被消费
- 6: **if** 存在 (x, i) 的最终创建 **then**
- 7: 对照声明检查其区块体与证据
- 8: **else**
- 9: 要求 $i = 0$ 且存在匹配的直接 TXCer c
- 10: 若尚未见过, 则登记 c 的全部覆盖
- 11: 创建唯一的 O_x ; 增加缺口与 Pending
- 12: **end if**
- 13: 在 O 中记录 (x, i) 的消费
- 14: **end for**
- 15: 要求 CAL 输入总额与输出总额相等
- 16: 记录 T 为 settled; 推进其费用阶段
- 17: 处理每个输出: 履行 Open 义务; 对 Repaired 义务向发行方偿还并将其标记为 Recovered; 否则履行任何已登记的 Promise。除非已偿还, 否则创建最终的实例 0
- 18: 将所有变更与结果作为一个转换返回

图 2. 公开支付执行。任何检查失败都会拒绝所有临时变更。输出处理遵循第 IV-C 节。

第二个输出: 金额返回到付款的发行方账户, 义务变为 Recovered, 偿还计数器与 grant 的 Spent 同时变化, 而赔付决定保持原有记录。已被消费的实例 0 在每个分支中都保持已消费; 被偿还的输出没有最终创建。来源交易自身的输入可能在同一原子转换内针对其直接发行方产生新的义务。

成员根据经认证的连续区块和结果重建进度, 且只有已存在的本地批准才提供待恢复的扣减。对其 CAL 扣减, 成员 m 计算

$$t_{m,T} = \begin{cases} a_T - (\hat{p}_{m,T} - \widehat{\text{Rec}}_{m,T}), & \text{若 settled,} \\ 0, & \text{否则,} \end{cases} \quad (3)$$

$$\delta_{m,T} = t_{m,T} - \text{Applied}_{m,T},$$

其中 “settled” 表示该成员已观察到自己的支付成功执行, $\hat{p}_{m,T}$ 与 $\widehat{\text{Rec}}_{m,T}$ 是为该批准的输出在本地记录的赔付与偿还。在观察到其支付的执行时, 成员检查经认证结果中的偿还额覆盖 $\hat{p}_{m,T}$, 并令 $\widehat{\text{Rec}}_{m,T} = \hat{p}_{m,T}$, 因为该次执行偿还了 T 的每个已赔付输出; 在跟随赔付之后才批准 T 的成员则未记录任何赔付。在检查边界与单调性之后, 它将 $\delta_{m,T}$ 在原 Worker 中从 Reserved 移至 Available 并更新 Applied。Pending 输入不影响 CAL

释放，因为这些责任由其发行方承担，而 work 资源则等待费用闭合。未执行的已认证来源在每个批准成员处保持预留其全部 CAL 金额（含找零）；仅有赔付不会释放它。已花费的 FUEL 不会作为未花费的费用被恢复。重复的区块与迟到的材料不能重复贷记。钱包将被偿还的输出记录为终态观察，其没有证书、最终创建或余额。

经授权的备付金增加从外部的、未受保护的 CAL 账户扣款，贷记备付金，并以原子方式提高其 grant。每个成员加上新旧 $[2B_g/3]$ 份额之差，同时 Reserved 及其批准保持不变，汇总的支持检查覆盖共享同一账户的所有 grant。

被取代的部分批准。公式 (3) 规范正常的结算与偿还。另有一个独立的本地终态转换，适用于以下情形：同一组织配置下经认证且成功应用的公开支付，消费了成员曾为另一个事实锁定的普通输入实例或用户支付的费用输入实例。在替换本地花费候选之前，跟随者重新加载不可变的批准，并检查确切的输出实例、不同的事实与交易，以及不存在已观察到的执行、已安装或已排队的证书材料、待处理义务、赔付、恢复或任何费用效果。

随后，成员对每个已记录的本地扣减 d ，在原资源切片中恰好将 $d.Cap - Applied$ 从 Reserved 归还到 Available，并将该批准记录为被第一个成功的冲突事实及其高度所作废。它不设置 Settled，不关闭费用托管，不修改批准，不清除其他输入锁，也不改变公开状态。累计的 Applied 向量使该转换在同一批准的多个输入发生冲突或区块被重放时保持幂等。重试与迟到的 INSTALL 被拒绝。将已作废事实作为所谓已认证来源而暴露，被视为会计矛盾，并在所述故障模型下使跟随者停止。超时、仅 INSTALL，或不相交输入之间的全局 Intent 冲突，均不授权此释放。

D. 基于公开证书的来源重新提交

执行某笔支付的区块携带认证其已认证输入及其创建支付的法定人数证书。每个成员在其经认证的区块事务中跟随该区块。对于所携带的、属于其自身组织的每份证书，它加载以该证书的事实为索引的批准，除非它已观察到该来源的公开执行，或已持有该事实的发件箱条目，并检查所存储的交易与准入向量能复现该事实。随后它组装所存储的交易、所存储的直接输入证书和暴露的 QC，并应用普通的支付验证。成功时，它在同一个原子本地更新中写入一个发件箱条目，并按成员索引错开首次尝试；既有的中继在提交后提交该条目。

其他组织的支付触发同一路径，而未通过这些检查的已存储记录属于损坏状态，会使跟随者停止。

该作业重新提交该组织所认证的支付，因此不需要新的所有者或成员签名，不创建批准，保持输入锁不变，也不预留额度。已存在的发件箱条目保留其重试时间，来源交易的执行会移除该条目。该路径需要持有该批准、跟随链并能到达委员会的原签名者，也需要不存在阻止该来源执行的全局 intent 冲突；截止期时仍缺失的来源交由赔付处理。它不提供固定时间保证。第 V-B 节表明，只要义务已公开，至少有两名诚实签名者持有该材料。

E. 赔付决定

截止期之后，Open 义务通过公开的 $CompensationDecision \delta = (net, x, h, j, k)$ 关闭：它由网络、被消费的输出，以及消费支付的高度、交易索引和输入索引构成。定长的规范编码既不含提交者签名，也不含适配数据。权限来自公开义务，且已提交状态固定目标坐标，因此对于给定的义务，每个提交者构成相同的字节。委员会 worker 扫描到期索引并提交到期的决定，而无需请求适配份额。

图 3 给出该转换。执行器在读取任何历史区块之前，先将请求的网络、位置和输入槽与义务比较，因此不匹配的请求无需加载历史即被拒绝。随后它读取位于 (h, j) 的规范消费支付，检查其身份、事实、输入和被声明的金额，并要求 x 具有 OriginalFunding、有效的开口以及同宽度的备付替换；它不计算替换开口，也不联系适配器。一个 overlay 承载所有变更：备付扣减、Repaired 状态、Paid、Spent、缺口、消费方的 Pending 计数、赔付费用（从消费方托管到奖励），以及在没有剩余输入义务时的消费方费用闭合与退款，连同不可变的决定记录 D_x （义务快照与备付扣减标识）和待表示条目。

结果携带单一的逐输出效果（它指明父事实与消费方事实、金额、输入位置和扣减标识）以及任何费用输出。重复的相同决定返回一个变更集为空的未应用结果（在来源偿还之后亦然），与记录不同的决定则被拒绝；一旦来源已执行，针对其义务的新决定不再提交。成员连同其游标一起应用经认证结果中的效果，并定位原始批准以进行 Paid 与 Pending 更新，钱包对每个费用输出只应用一次。赔付不向接收方或已执行的后继交易计入任何本金。

Require: 决定 $\delta = (net, x, h, j, k)$, 状态 S , 执行高度与区块时间

- 1: 要求 h 早于执行高度
- 2: 若 $\mathcal{D}_x$ 存在, 则当其等于 δ 时返回无效果; 否则拒绝
- 3: 要求 $\mathcal{O}_x$ 为 **Open**, 消费方位置为 (h, j, k) , 网络为 net
- 4: 要求 $\mathcal{O}_x$ 的截止期已被区块时间到达
- 5: 加载位于 (h, j) 的规范消费支付; 检查其身份、事实、输入和被声明的金额; 要求 x 具有 **OriginalFunding**、有效开口以及同宽度的备付引用
- 6: $O \leftarrow \text{Overlay}(S)$
- 7: 在 O 中: 扣减发行方备付金; 将 $\mathcal{O}_x$ 设为 **Repaired**; 将其 **Promise** 转为 **Paid**; 更新 **Spent**、缺口以及消费方的 **Pending** 计数和费用阶段
- 8: 在 O 中: 记录 $\mathcal{D}_x$ 和待表示条目
- 9: 返回变更与赔付结果

图 3. 赔付决定。该转换不读取也不请求适配材料。任何检查失败都会拒绝所有临时变更。

F. 表示命令与批次

一个 **RepairBatch** 针对一个历史区块, 按规范顺序含 1-32 个条目, 每个条目指明一个输出、一个交易索引、一个输入索引和一个新的输入开口; 输出与槽位唯一。该批次固定基础修订以及先前与替换后的规范区块体摘要, 携带 **part** 开口, 并将逻辑修订从 **Base** 推进到 **Base + 1**。单个 **RepairInput** 是其单条目形式, 并携带被替换的交易字节。两种命令都只接受具有已提交赔付决定的输出, 从决定记录中获取金额、发行方和替换引用, 并且不写入会计状态。

公开执行对照其当前状态重复所有检查。基础已变化、某条目没有决定, 或某槽位不再持有 **OriginalFunding**, 都会拒绝整个批次, 且确切的成功重放在基础验证之前即被识别。由于每条命令都要求其槽位中为 **OriginalFunding**, 不同的批次或单条目入口不能对同一槽位替换两次, 命令与修订标识保持特定于所选的打包方式。经认证的结果指明该命令及其是否被应用; 跟随者将其映射为空的经济效果, 而赔付的经济效果仅来自决定结果。偿还属于来源交易自身的执行结果。

G. 保持身份的安装与重放

四种身份被区分开。支付身份 $\text{id}(T)$ 固定授权与承诺, 且所有者签名绑定它; 它不是共识交易哈希。格式良好的 **UTX04CH** 信封的共识 $Tx.Hash$ 是对其头部与固定字段的 **SHA-256**, 不检查任何支付有效性。 $DataHash$

是共识交易哈希之上的 **Merkle** 根, 而包含性证明不授予修订权限。完整 **BlockID** (H_b, P_b) 保留区块哈希与原始 **part-set** 头, 执行与安装都比较这两者。**Part** 承诺是绑定到链、密钥、高度和 **part** 索引的变色龙承诺, 被更改的 **part** 携带指向其原始承诺的有效开口。原始共识 **part** 携带 **revision 0** 与 **opening 1**; 原始验证要求正确的高度与这些值, 且应用正常验证原始支付。被修订的 **part** 不能冒充新的原始执行, 只有已提交的 **Task** 才授权改写。重放使用保留的原始命令, 且从不会被修订的区块体作为新支付执行。

公开的修订与 **Task** 授权一个确切的新区块体及其 **part** 证据, 安装器在获取物理修订锁之前, 从一个已提交的读视图中复制它们。对于目标修订 r , 较低的物理版本被验证并安装, 相等的版本要求材料相同, 较高的版本则不经回滚而覆盖旧任务。原始区块被保留, **part** 与本地版本元数据被原子地写入。安装可以跳过中间的物理版本, 并保留每条经济命令。队列进度保持全局且连续: 对于条目 A_1, B_1, A_2 , 在处理 A_1 时安装 A_2 会使 B_1 保持未完成。逻辑执行读取已提交的 **Canonical** 修订, 因此每种安装调度都留下相同的经济历史。当某个修订的区块体已在某副本上被物理安装, 该修订即在该副本处被物化。

重放按已提交的顺序执行保留的原始区块, 包括支付、赔付决定和表示命令, 并重建该序列的应用哈希。既有的后继交易保持其授权、输出引用和区块身份。仅赔付决定就以仅追加的公开记录在经济上关闭义务。表示增加了一个接口: 在消费方的原始坐标 (h, j, k) 处, 已存储的区块指明指定输入的备付扣减, 并仍能对照原始 **BlockID** 验证。读取者仍读取永久决定与义务状态以获得经济状态, 被修订的字段并不取代它们。批处理在一个区块内共享表示工作, 直接安装合并历史写入, 二者都不为正常认证增加一轮交互。第 **VI-C** 节报告了该接口相对于优化后决定索引的读取收益以及存储和维护代价。

H. 在原始坐标处读取资金引用

位于 (h, j, k) 的读取返回 $(H, h, j, k, F, r, d, s_H)$, 由现有函数在 V_H 上的一个读事务内组装而成。其中 F 是资金表示, r 是其修订, d 是决定或缺失标记, s_H 是义务状态。该逻辑元组描述本地组合, 而不是已部署的网络接口。索引 j 对原始交易列表计数, 不会因过滤维护命令或失败交易而重新编号。逻辑区块体来自 V_H 中

已提交的修订，仅当不存在修订时才回退到原始区块，因此物理安装进度，或重放期间已持有较晚版本的区块存储，都不改变答案。存在但无法解码的修订是错误，绝不回退到原始区块。缺失的历史被报告为不可用，而不被解读为不存在赔付；该契约假设相关历史未被裁剪。在完整、已验证、未裁剪的 V_H 中，缺失决定意味着截至 H 不存在决定。

四个状态维度。这些维度描述不同的对象并各自独立推进。

维度	含义
公开支付	Settled 记录成功执行。新创建的输出即使在 Pending 非零时也是最终的；具有 Repaired 义务的输出向备付金偿还，而不是被重新创建。
直接义务	Open → Fulfilled , 或 Open → Repaired → Recovered 。 Repaired 表示赔付； Recovered 表示随后的偿还。赔付决定持续存在。
逻辑表示	V_H 中的修订决定该槽位。 OriginalFunding 并不意味着不存在赔付。 ReserveFunding 记录一次历史垫付，并在偿还之后仍然保留。
物理安装	ρ_p 记录已安装的区块存储版本。安装进度不决定 H 处的逻辑答案，也不改变经济状态。

因此，读取者将永久决定与义务状态连同资金表示一并查阅。待表示条目是可移除的工作状态，而不是赔付已发生的持久证据。这些表示或安装变更都不会向消费方计入第二笔本金。

可执行的归档消费者。一个本地示例在 (h, j, k) 处导出原始与授权后的区块体字节、完整 BlockID、Revision 与 Task 标识符、历史资金引用以及当前义务状态。它检查确切的 Task 区块体、重建的 part、所有者授权、QC 和输入开口。偿还之后，授权字节保持不变，而义务变为 **Recovered**。该字节导出契约是对经济元组的补充；永久决定仍然必要。该示例在已验证、未裁剪的副本上运行，不是远程证明协议。

密码学实例化。该实现使用 RSA 变色龙哈希，模数为 2048 位， $e = 65537$ ，并使用带上下文绑定的 SHAKE256 全域哈希映射到 $\mathbb{Z}_N^*$ 。一个可信 dealer 使用 CIRCL v1.6.5 [19] 生成三选四的门限 RSA 密钥，遵循 Shoup 的门限构造 [15]。公开的适配比值不传递编辑权限；传递编辑权限的是已提交的决定、确切的 Task 以及完整身份守卫。补充材料规定了编码、参数假设和原始群适配接口。

V. 安全性分析

我们在第 III-A 节的模型下，分析第 IV 节协议的有限执行。论证围绕转移函数的守卫条件展开；除故障与状态假设外，还使用第 IV-A 节的验证规则：输入证书绑定网络、组织、事实和声明的输出，并与证书型输入一一对应；每个准入向量均包含一个绑定至签发者的 CAL 条目。表示只能通过已提交的命令生效，该命令的守卫要求已有补偿决定，据此导出槽位和储备引用，并验证每个非目标字节及完整区块身份。因此，适配输出只是候选材料，其可用性仅影响表示的进展。共识论证将 Tendermint 锁定接口 [20] 特化到完整区块身份。

A. 原始执行与认证消费

一个 *polka* 是同一轮次中针对某个完整区块身份或 nil 的、达到法定人数的预投票集合。

引理 1 (原始执行的一致性): 在共识接口的锁定与有效性规则下，每个已提交高度仅接纳一个原始区块体，包括其应用上下文。

证明: 当原始开口为 1 时，部件叶节点对其绑定上下文的消息代表元求哈希，因此绑定性和 Merkle 证明在原始部件集头下固定了该部件的字节。候选区块体从部件重建，或在本地与部件配对；Locked 和 Valid 的转移保持这种配对，提案、锁定证明 (POL)、锁定和提交的比较均使用 (H_b, P_b) ；下载目标变化时清除旧区块体，轮次与步骤守卫则确保每轮只发出一次预提交。

同轮次的冲突提交需要诚实成员发出双重预提交。取值 v 在轮次 r 的一次提交及其两名诚实签署者，并在随后出现冲突 *polka* 的轮次中选取最小轮次 s ，以及该轮最早形成证书的事件；这里也计入允许解锁的 nil *polka*。该法定人数集合与这两名签署者的集合相交，而交集的签署者要发出该投票，就需要更早的解锁依据，这与 s 或证书形成事件的选取矛盾。对锁定值的投票可沿更早的锁定轮次追溯至提案验证，Finalize 则在与完整区块身份匹配的提交之后执行。同步与重放保持区块体、高度和时间不变，补充材料进一步展开了这一论证。 ■

引理 2 (经过认证的唯一消费): TXCer 固定其输出和签发者。不存在两张有效证书授权对同一 (x, i) 的冲突消费，且公开执行至多消费该实例一次。

证明: 绑定性固定认证摘要和所有者授权的交易，固定路由将竞争请求送至同一组织。两个三成员法定人数集合至少交于两名成员，因此交集中存在诚实成

员。该成员在签名前原子地锁定输入，所以两个冲突请求中只会批准一个，且预算释放不会清除该锁。最终输入和证书型输入的查询同一个消费键。重放一个事实既不增加消费，也不增加费用扣收。已偿还输出没有最终创建记录，其实例 0 保持已消费状态，因此之后针对 (x, i) 的声明会失败。 ■

引理 3 (被取代的部分批准): 固定一个四成员组织，要求三份签名，且至多有一名拜占庭成员。设某诚实成员已批准事实 c ，并锁定了普通输入或用户付费输入实例 ξ 。如果同一组织配置下一笔成功应用的公开付款携带针对另一事实 c' 的有效 QC，具有不同的交易身份，并消费了 ξ ，那么不可能曾经存在针对 c 的三份相匹配的有效成员签名。因此，该诚实成员可以恢复 c 的剩余本地扣减额，同时永久拒绝签署或安装 c 。这一结论不适用于输入互不相交、仅因全局 Intent 而冲突的两个持有 QC 的事实。

证明: c' 的公开执行已通过当前组织的授权检查，因此携带有效的三成员 QC。反设 c 也具有有效的三成员 QC。两个签署者集合至少交于两名成员。由于至多一名成员是拜占庭成员，交集中必有一名诚实成员。这意味着该成员签署了两个消费同一实例 ξ 的不同事实，然而诚实成员在签名前会原子地锁定普通输入和用户付费输入实例，并拒绝针对已锁定实例的另一候选请求。这与批准规则矛盾。

因此，不可能曾经存在针对 c 的三份相匹配的签名。使用第 V-C 节引入的记号，有 $c \notin C_g$ 。恢复某成员对 c 的剩余扣减额，不会移除 Q_g 中任何一项的容量见证。对 ξ 的成功公开消费、不可变的批准记录以及标记为失效的进度记录，会继续拒绝之后对 c 的使用；恢复的仅是其尚未使用的本地资源预留。 ■

B. 源交易重提交与补偿决定

对于输出 x ， $\mathcal{O}_x$ 表示其直接义务，状态为 Open、Fulfilled、Repaired 或 Recovered， $\mathcal{D}_x$ 表示其不可变的补偿决定。Repaired 表示已经完成经济补偿，与存储字节的状态无关。决定在 Open 向 Repaired 的转移中写入，且永不删除，因此 $\mathcal{D}_x$ 存在，当且仅当 $\mathcal{O}_x$ 处于 Repaired 或 Recovered。集合 $\mathcal{R}$ 保存表示状态，包括已提交修订版本、头命令、任务、批次引用、队列和待表示索引；投影 π_{econ} 保留公开资产、消费、覆盖、义务、授权项用量和费用状态，而省略 $\mathcal{D}$ 和 $\mathcal{R}$ 。

引理 4 (重提交保持授权不变): 设一个诚实成员为事实 c 创建重提交任务。该任务提交成员在签署 c 之前存储的付款、准入向量和直接输入证书，并附上 c 的公开 QC；公开验证与执行将其视为该付款的普通提交。它不需要新的所有者或成员签名，不创建批准，不改变输入锁，也不预留签名容量。每个成员对每个事实至多保有一个任务，付款执行后移除该任务。一旦 c 的某个输出上的直接义务已经公开， c 的至少两名诚实签署者就保有这些存储材料。

证明: 成员使用公开 QC 中的事实索引其批准记录，存储的付款和准入向量必须重现该事实。它将这些材料、存储的直接输入证书与已公开的 QC 组合，并在创建 outbox 条目之前执行普通付款验证。事实绑定与法定人数验证排除了其他付款核心，法定人数签名则固定了公开方选择携带的证书内容。该转移只写入 outbox，因此批准、输入锁和授权项预留保持原样；已有 outbox 和已观察执行标记抑制重复。未通过这些检查的存储记录不属于故障模型允许的状态。

义务只能由消费认证缺失输入的已执行付款产生，而公开验证要求提供该输入的证书，因此该证书必然已经公开。其三名不同签署者中至少有两名诚实成员，每名诚实成员都在返回签名之前存储了批准材料。批准记录本身不包含法定人数证书，后者随区块到达，因此客户端扣留证书不会破坏重提交所需的材料。执行还需要持续工作的诚实跟随者、公平的消息交付，以及满足普通执行规则的源付款，其中包括全局 Intent 检查。 ■

引理 5 (证书限定责任，而非保证执行): 设 $T_1 \neq T_2$ 是具有相同 Network、Subject 和 Nonce 的两笔付款，分别消费路由至组织 G_1 和 G_2 的互不相交输入，且各自满足所属组织的批准规则。两笔付款都可以获得 QC，但在任何公开顺序中至多有一笔执行。恶意用户可以造成这种情形，重提交其中任意一笔都不会消除冲突。不同 Subject 的付款不会仅因使用相同 nonce 而冲突。

证明: 各组织维护各自的本地 Intent 记录，且批准涉及互不相交的输入，因此两个组织都不会观察到冲突，而所有者授权涵盖每笔付款。在公开执行时，如果 Intent 已绑定至不同的 TxID，求值过程就返回冲突。该绑定在成功执行付款的同一转移中写入，且不存在删除或重新绑定它的生产路径，因此无论怎样重提交，另一笔付款都保持未执行状态。不同 Subject 产生不同的 Intent。 ■

如果 T_2 无法执行，但消费其认证输出的付款已经

执行, 该消费方就登记了一项针对 G_2 的义务。补偿扣减该签发者的资金, 但不会使 T_2 变得可执行。其批准成员保持 $r_{m,c} = a_c$, 责任也保持为 $w_c = a_c$ 。原输入继续锁定, 但控制消费方输出的用户可以在后续请求中使用这份新价值。重复这一过程需要新的成对 Intent、可用路由、认证容量和费用; 最终后代输出也必须保有可用路由。这里复用的是后代输出的价值, 而不是解锁被拒绝源交易的输入。在用户付费模式下, 被拒绝源交易的完整 FUEL 输入仍保持锁定, 而不只是其签名授权的托管上限。定理 3 排除了这一情形, 因为并非每笔付款都成功。终止对冲突源交易的重复提交, 并不会授权释放其由证书承载的责任。

引理 6 (补偿至多生效一次): 每项直接义务至多发生一次储备扣减及其相关经济效果。如果 x 已有记录 D_x , 与该记录相同的决定不产生效果, 偿还之后也如此; 不同的决定则被拒绝。如果尚无记录, 一旦 O_x 不再是 Open, 决定就被拒绝。

证明: 状态只沿 Open→Fulfilled、Open→Repaired (仅由决定触发) 和 Repaired→Recovered (发生在源交易成功执行之内) 变化。决定首先检查 D_x : 完全匹配时返回未应用结果和空变更集, 不匹配时拒绝。没有记录时, 转移要求义务处于 Open、区块时间已达到截止期, 并且历史位置、金额和资金承诺相匹配; 这些内容都从已提交状态重新导出。随后, 它在一个原子步骤中扣减签发者储备、关闭缺口、更新费用与覆盖, 并记录 D_x 。状态机无环, 因此只会离开 Open 一次, 至多有一个决定生效。

如果源交易先执行, 义务就离开 Open, 不会再有针对它的决定提交。如果补偿先发生, 源交易执行时通过唯一的 Repaired→Recovered 转移将资金返还签发者, 保留 D_x , 且不创建用户输出。对公开命令顺序归纳, 即得到补偿和偿还均至多发生一次。 ■

C. 公开与隐藏承诺的覆盖

每个认证事实都携带一个绑定至签发者的 CAL 条目, 因为至少两名诚实签署者在签名前按照准入规则计算了向量, 且事实绑定该摘要。对于完整的 CAL 授权项 g , 累计授权额 B_g 为每名成员提供 $s(B_g) = \lfloor 2B_g/3 \rfloor$ 的容量。令 C_g 包含截至当前前缀曾经存在三份签名的所有不同事实, 包括对应尚未组装或隐藏证书的事实, 以及已结算事实。唯一的 CAL 分配项将每个事实绑定至一个配置、ResourceKey 和 Grant ID。事实 c 的输出

上限为 a_c , 即其所有 CAL 输出之和。对于 $c \in C_g$, 令 p_c 为针对这些输出累计支付的补偿额, Rec_c 为源交易执行时偿还的部分, z_c 表示 c 对应的付款是否已成功执行。净补偿额为 $\bar{p}_c = p_c - \text{Rec}_c$ 。定义

$$w_c = \begin{cases} a_c, & z_c = 0, \\ \bar{p}_c, & z_c = 1, \end{cases} \quad Q_g = \sum_{c \in C_g} w_c. \quad (4)$$

引理 7 (源交易偿还): 已补偿输出仅在其源交易成功执行之内离开 Repaired 状态, 且该过程与设置 $z_c = 1$ 发生在同一转移中, 偿还资金记入原先被扣款的账户。因此 $0 \leq \text{Rec}_c \leq p_c \leq a_c$; 当 $z_c = 0$ 时, $\text{Rec}_c = 0$; 一旦 $z_c = 1$, 就有 $\text{Rec}_c = p_c$ 。特别地, 源交易尚未执行时 $\bar{p}_c = p_c$, 执行之后 $\bar{p}_c = 0$, 因此源交易执行后不再留下净补偿额。

证明: 每项已登记承诺至多离开 Open 一次; 根据引理 6, 每个输出的补偿至多生效一次, 且仅在其义务处于 Open 时生效, 因此 $p_c \leq a_c$ 。唯一离开 Repaired 的转移, 是某笔付款的输出处理步骤, 该付款的事实和签发者必须等于义务所记录的证书事实和签发者。该步骤在将付款记录为已结算的同一覆盖层中执行, 因此 z_c 与偿还同步变化。它处理源交易中每个具有义务的输出, 将 Open 转为 Fulfilled, 将 Repaired 转为 Recovered; 之后 c 的每项承诺均已关闭, 因此不会产生新的补偿, 重复提交也为空操作。补偿扣款和偿还入账均使用义务中存储的签发者账户, 即 c 的单一 CAL 分配项所绑定的账户, 且偿还不改变 D_x 。 ■

对于任意本地批准, 令 $r_{m,c} = a_c - \text{Applied}_{m,c}$, 其中 Applied 是其累计 CAL 恢复额。

引理 8 (残留容量): 对于每个事实 $c \in C_g$, 以及批准了 c 的每名诚实成员 m , 在每个前缀都有 $r_{m,c} \geq w_c$ 。

证明: 对于 $c \in C_g$, 引理 3 排除了诚实签署者将其置为失效的可能性: 触发失效的冲突会蕴含 $c \notin C_g$ 。在 m 观察到 c 对应付款成功执行之前, 其对 c 的 CAL 恢复额为零, 因此无论补偿状态如何, 都有 $r_{m,c} = a_c \geq w_c$ 。观察到执行之后, m 检查报告的偿还额是否覆盖自身记录的补偿额, 并将记录的偿还额设为该补偿额。恢复目标为 a_c 减去记录中为零的净补偿额, 因此 $r_{m,c} = 0$; 由引理 7, 有 $w_c = \bar{p}_c = 0$ 。在跟随补偿事件之后才批准的成员没有记录该补偿, 只恢复自身的扣减额。 ■

定理 1 (潜在责任具有资金覆盖): 对于每个 CAL 授权项, 都有 $Q_g \leq B_g$ 。对于每个储备账户 x , 其未闭缺口总额 D_x 不超过账户余额 A_x 。

证明: 令 $\mathcal{A}_{m,g}$ 包含 g 下的全部成功批准, 包括没有 QC 的批准 (失效批准的残留贡献为零), 并令 $L_{m,g} = \sum_{c \in \mathcal{A}_{m,g}} r_{m,c}$, 该求和涵盖各 Worker。批准、普通恢复和被取代后的失效处理在 Available 与 Reserved 之间移动容量, 补资则增加成员份额的差值, 因此对于本地观察到的授权额 $B_{m,g}$, 有 $L_{m,g} \leq s(B_{m,g}) \leq s(B_g)$; 授权额只增不减。固定该组织中任意三名诚实成员组成集合 H_g 。QC 有三名不同签署者, 而 H_g 之外至多只有一名成员, 因此 H_g 中签署了 c 的成员集合 Sig_c 至少含有两名成员。诚实成员仅在批准之后签名, 所以由引理 8, 对 $m \in \text{Sig}_c$ 有 $r_{m,c} \geq w_c$, 并且

$$\begin{aligned} 2Q_g &\leq \sum_{m \in H_g} \sum_{\substack{c \in C_g \\ m \in \text{Sig}_c}} r_{m,c} \\ &\leq \sum_{m \in H_g} L_{m,g} \leq 3[2B_g/3] \leq 2B_g. \end{aligned} \quad (5)$$

令 R_c 和 d_c 分别为已登记的剩余覆盖额和已解除覆盖额, 登记前有 $p_c = d_c = R_c = 0$ 。已登记记录满足 $a_c = p_c + d_c + R_c$, 且 $z_c = 1$ 蕴含 $R_c = 0$ 。因此 $R_c \leq w_c - \bar{p}_c$: 当 $z_c = 0$ 时, $\text{Rec}_c = 0$, 所以 $R_c \leq a_c - p_c = w_c - \bar{p}_c$; 一旦 $z_c = 1$, 两边都为零。定义 $S_g = \sum \bar{p}_c$, $V_g = \sum R_c$ 和 $E_g = Q_g - S_g$, 各求和均遍历 C_g 。登记、补偿、偿还和覆盖解除共同维持公开状态中的 $\text{Usage.Spent} = S_g$ 和 $\text{Usage.Reserved} = V_g$ 。

令 $\mathcal{G}_x$ 包含全部由 x 提供资金支撑的授权项; 其认证的 CAL 账户就是被扣款的签发者账户。创世状态建立 $\sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x$ 。来自外部非受保护账户的补资使两边等量增加。补偿使 S_g 增加, 并使 A_x 减少相同金额; 偿还使 S_g 减少, 并使 A_x 增加相同金额 (引理 7); 不存在其他提取 CAL 储备的协议操作。每项 Open 义务对应一项不同的剩余承诺, 因此

$$\begin{aligned} D_x &\leq \sum_{g \in \mathcal{G}_x} V_g \leq \sum_{g \in \mathcal{G}_x} E_g \\ &\leq \sum_{g \in \mathcal{G}_x} (B_g - S_g) \leq A_x. \end{aligned} \quad (6)$$

累计补偿总额 Paid 及其偿还均只计入一次, E_g 是风险上界, 而不是成员的可用容量。 ■

部分批准持续占用额度, 直到其自身付款执行, 或经过认证的公开历史证明其锁定的某个实例已被同一配置下另一个成功获授权的事实消费。时间本身绝不释放该预留。一笔输入互不相交、在全局 Intent 冲突中失败的付款, 仍可能具有有效 QC 和签发者责任, 因此保留其输入锁和签名预留。

CAL 准入推论。在执行前状态中, 包括同一区块内更早成功命令产生的状态, 令 N_g 为 g 下相匹配且尚未消费的缺失输入所需的、不同且有效而尚未登记的父事实集合。它们满足 $w_c = a_c$ 和 $p_c = R_c = 0$ 。每个已登记事实满足 $R_c + \bar{p}_c \leq w_c$, 且这些事实互不重叠, 因此

$$V_g + S_g + \sum_{c \in N_g} a_c \leq Q_g \leq B_g. \quad (7)$$

同一证书的重复使用只计一次, 顺序登记能够看到此前的预留, 因此每个部分和都满足该上界。故首次登记不可能因 CAL 授权容量不足而失败, 即使它先于同一转移中的偿还执行也是如此, 因为偿还只会降低 S_g 。同样, 金额为 $a \leq D_x \leq A_x$ 的 Open 义务具有足够的储备本金。其他资源和费用检查仍是独立要求。

D. 资产守恒与延迟中立性

定理 2 (CAL 守恒): 令 U 为最终且未消费的 CAL 实例的总价值, A 为 CAL 账户余额总和, D 为 Open 缺口的总价值。每个执行前缀均满足

$$U + A - D = C_0, \quad (8)$$

其中 C_0 包括创世输出和所有账户, 也包括补资金来源账户。

证明: 一笔付款消费最终价值 F 和缺失价值 M , 并产生 $F+M$ 。令 J 和 R 分别为其到达输出中、义务原先处于 Open 和已补偿状态的总价值。前一类输出已经被消费, 因此它们关闭缺口而不增加未消费价值; 后一类输出不生成创建记录。于是 $\Delta U = M - J - R$, $\Delta D = M - J$, 偿还使 $\Delta A = R$, 从而 $\Delta(U+A-D) = 0$ 。金额为 a 的补偿决定使 $\Delta A = \Delta D = -a$, 且 $\Delta U = 0$ 。重提交不改变公开状态; 由引理 10, 表示命令和安装保持 U 、 A 和 D 不变。补资在已计入的账户之间转账, 其他转移也保持该投影。 ■

FUEL 单独记账: 每笔托管上限等于 Held 加上累计 Rewards、Burned 和 Refunded; 全局守恒总量计入未消费 FUEL、普通账户、当前 Held、奖励余额和累计销毁额。唯一的阶段身份和义务身份使价值仅转移一次, 对用户付费和组织付费均如此。

守恒确定总量, 而下面的结果确定最终持有者。

定理 3 (CAL 延迟中立性): 令 $\mathcal{T}$ 为有限付款集合, 满足: (i) 任意两笔付款不消费同一实例或费用输入; (ii) “消费由另一付款创建的输出”这一关系无环; (iii) $\mathcal{T}$ 中付款的每个证书型输入均认证 $\mathcal{T}$ 中某笔付款的输

出。令 $\mathcal{U}$ 为有限的已授权储备增加集合。从干净的创世状态开始，其中不存在覆盖、承诺或义务记录， CAL Reserved 和 Spent 均为零。令 ρ_0 按照符合条件 (ii) 的顺序执行 $\mathcal{T}$ ，并应用 $\mathcal{U}$ ；令 ρ 为任意执行，由 $\mathcal{T}$ 中的付款、 $\mathcal{U}$ 中的增加以及到期 Open 义务的补偿决定以任意顺序组成，并可穿插符合规则的表示命令。假设每笔付款和每次增加在两个执行中都恰好成功一次。则在 ρ 和 ρ_0 结束时：

- 1) 最终且未消费的 CAL 实例相同，所有者和金额也相同；
- 2) 每个储备账户具有相同的 CAL 余额；
- 3) 每个 CAL 授权项具有相同的授权额，且 $\text{Reserved} = \text{Spent} = 0$ ；每项义务均为 Fulfilled 或 Recovered ，并且每张证书都满足 $\text{Paid}_c = \text{Rec}_c$ 。

不比较费用和 FUEL 。

证明：持有者。由条件 (i)，两个执行消费相同的实例。在 ρ 中，一个在源交易执行之前被消费的输出会获得一项义务；源交易执行时，该输出要么转为 Fulfilled ，具有对应实例已经被消费的创建记录；要么转为 Recovered ，没有创建记录；两种情况下它都被消费。未被 $\mathcal{T}$ 中任何付款消费的输出，在两个执行中都被创建为最终且未消费的输出。因此，未消费实例就是创世输出和 $\mathcal{T}$ 的输出中未被 $\mathcal{T}$ 消费的那些实例。

储备。对于储备账户 x ，令 $\hat{A}_x$ 为其余额加上由 x 签发的 Repaired 义务金额之和。补偿降低 A_x ，并使 Repaired 总额增加相同金额；偿还在同一账户上执行相反变化（引理 7）；创建或履行义务则不改变这两项。与定理 1 相同，只有补资、补偿和偿还改变储备余额，因此每个前缀都有 $\hat{A}_x = A_x(0) + \tau_x$ ，其中 τ_x 是截至该前缀记入 x 的增加总额。义务仅针对在源交易执行之前被消费的输出存在，而根据条件 (iii)，该源交易会在 ρ 中执行，使义务变为 Fulfilled 或 Recovered 。结束时不存在 Open 或 Repaired 义务，因此 $A_x = A_x(0) + \tau_x$ ，这也等于 ρ_0 中的值。

用量。每张已登记证书都被某个缺失输入使用，因此由条件 (iii)，其源交易会在 ρ 中执行。该执行关闭其全部承诺，所以 $\text{Remaining} = 0$ ，而引理 7 给出 $\text{Paid}_c = \text{Rec}_c$ 。因此 $\text{Reserved} = \sum \text{Remaining} = 0$ 且 $\text{Spent} = \sum (\text{Paid}_c - \text{Rec}_c) = 0$ ，授权额仅通过 $\mathcal{U}$ 改变。

表示。根据引理 10，穿插的表示命令不会改变任何被比较的量，也不会改变随后在相同高度和时间执行的付款或决定的结果。 ■

中间状态有所不同：在源交易执行之前，已补偿源交易的签发者持有的资金更少，定理 1 覆盖这种情形；比较所用的投影省略了仅存在于 ρ 中的决定记录和表示状态。永不执行的源交易会使其补偿金额成为签发者的净损失，激励问题不在该定理范围内。

E. 授权表示及其独立性

引理 9 (由决定约束的表示权限)：每个生效的表示项均引用一个已有补偿决定，包括其义务后来已获偿还的决定，并引用一个仍保存 OriginalFunding 的规范槽位。成功的表示不会改变获授权资金槽位之外的任何交易字节，即使适配输出由对手控制也是如此。

证明：令 N 为 RSA 模数， e 为公钥指数， r 为 $\mathbb{Z}_N^*$ 中的开口。经过域分离的代表元 $\text{FDH}(ctx, m)$ 绑定消息及其修复上下文。关系式

$$C = \text{FDH}(ctx, m)r^e = \text{FDH}(ctx, m')r'^e \pmod{N} \quad (9)$$

仅给出兼容性，不携带新的批准；已公开的开口比值可以在相同消息关系下复用。公开守卫读取保存义务快照和储备扣款身份的决定记录，并从中导出槽位与替换引用。它们检查当前基版本和区块体摘要，并验证全部非目标字节及完整区块承诺。读取快照使已决定的槽位在偿还之后仍可进行表示。无效候选不留下任何变更，对已成功命令的完全相同重放为空操作，而重新封装的命令会遇到不再保存 OriginalFunding 的槽位或已有任务。经济效果已由决定提交，因此表示命令不提供金额、签发者或账户变更，适配器也不写入账本状态；该论证仅依赖经过验证的输出。 ■

引理 10 (表示不产生经济影响)：对于每个成功的单项或批量表示命令 r 以及状态 S ，有 $\pi_{\text{econ}}(r(S)) = \pi_{\text{econ}}(S)$ 。在相同高度和时间执行的付款或决定，在 $r(S)$ 和 S 中产生相同结果，且两个后继状态的投影相同。

证明：表示命令检查决定、规范目标、基修订版本、前序摘要和获准的替换。其写集包含修订版本、头命令、任务和批次引用记录、队列以及待处理索引项的删除；它跳过记账转移，不写入任何账户、义务、覆盖、用量、消费或费用记录。跟随者将其结果映射为空经济效果。因此，单步不改变投影；通过归纳，任意此类步骤序列也不改变投影。

针对输出 y 的决定只通过交易身份、授权事实、输入、声明和金额、编码长度以及 y 的槽位读取规范区块体。如果 $\mathcal{D}_y$ 存在，该决定就是不读取历史的空操作。

否则, y 的槽位仍保存 `OriginalFunding`, 因为替换要求已有 D_y ; 替换其他固定宽度槽位不会改变其余字段。付款执行从不读取规范区块体。 ■

引理 11 (身份保持与安装独立性): 生效的表示保持所有者授权、交易和输出身份、声明、后继引用以及完整的原始 `BlockID` (H_b, P_b) 不变。对于同一公开历史, 合法的物理安装调度具有相同的经济投影。此外, 如果变色龙关系对于每个固定消息和承诺都具有唯一有效开口, 那么对于固定的原始区块体及固定的已决定槽位集合, 最终区块体和部件开口不依赖表示命令的分组或顺序。修订号、命令历史和应用哈希则可能不同。

证明: 执行器仅改变声明的资金槽位及其开口, 以构建规范区块体。式 (1) 的身份编码保持交易身份、声明和输入承诺不变, 替换也保持编码宽度不变。执行器验证新开口, 重新计算修订区块体的区块哈希和部件集头, 并且仅在它们与原始 `BlockID` 相等时接受。Base 和 Previous 将竞争的规范修订串行化, 已提交 Task 则绑定允许的字节。后继引用交易和输出身份, 因此既不需要重新签名, 也不需要重新执行。

Canonical 读取已提交的逻辑修订版本 ρ_ℓ , 精确的 Task 授权修订版本 $r \leq \rho_\ell$ 下的区块体和部件。若 $\rho_p < r$, 安装向前推进; 若 $\rho_p = r$, 检查完全相等; 若 $\rho_p > r$, 则视为已覆盖, 不进行回滚。安装不写入经济状态, 仅读取已提交的修订和任务记录, 队列进度也只越过已完成条目。对于同一公开前缀上的合法调度 μ_1, μ_2 , 有

$$\pi_{\text{econ}}(S_{\mu_1}) = \pi_{\text{econ}}(S_{\mu_2}). \quad (10)$$

收敛结论来自每个槽位和部件开口的唯一性, 以及部件上下文不依赖修订计数器。补充材料给出了这一论证, 以及对跟随者更新和历史的比较。 ■

读取方范围。 这些引理适用于在自身状态上执行过守卫检查的副本, 因此第 IV-H 节的读取方信任执行了这些检查的本地状态。一个区块即使携带 `RepairBatch`, 如果该命令未成功执行, 也不赋予任何表示权限。单凭包含证明、相等的 `BlockID` 或变色龙关系, 都不能证明某个表示截至 H 已获授权。若要服务于不可信的远程读取方, 就需要与认证状态根或原始执行证明绑定的授权证据; 本文未定义此类证据。

F. 受保护的连续支付与进展

令 `FreshRecv`(w, o, c) 表示诚实所有者经过认证的接收, 且该所有者此前未授权消费该实例。新鲜性是关

于历史的谓词, 不能仅由成功接收推出。

定理 4 (受保护的连续支付): 在 `FreshRecv` 之后, 未经所有者授权, 认证输出不能被重新解释或消费。已获认证并公开执行的后继要么消费最终源输出, 要么原子地建立一项具有资金覆盖的直接义务。源交易闭合、补偿和表示保持其已经成立的本金效果, 无需后继重新签名或重新执行。

证明: 引理 2 固定所有权并排除冲突消费, 原子执行则将每次缺失消费与其义务和输出绑定在一起。定理 1 提供资金覆盖, 定理 2 核算闭合与迟到源交易, 引理 1 确定已执行历史, 引理 4 约束重提交, 引理 6 约束补偿, 而引理 9–11 限制后续变更。对本地与公开转移组成的联合序列归纳, 可组合这些性质。INSTALL 保留本地消费效果, 不改变公开资产。补偿决定可以改变 Pending、费用和储备余额, 但不会重复产生后继本金; 表示和物理安装则不改变这些量。 ■

本地保留情形。 以下情形区分资源恢复与输入解锁。仅凭时间流逝, 二者均不获授权。

批准状态	现有规则下的结果
没有 QC 的独立部分批准	容量可能发生碎片化, 而不产生公开责任。具有实际资金支持的增加可以使付款完成, 但不会清除已有锁。
全诚实成员下的同输入 2/2 分裂	两个候选都无法获得第三份签名。增加容量不能消除冲突的输入锁。
被取代的未认证批准	同一配置下成功执行的冲突 QC 所对应付款, 证明旧批准不可能曾经形成 QC。满足守卫条件的失效处理只恢复其剩余本地扣减额。
持有 QC 的 Intent 失败方	输入互不相交时, 因 Intent 被拒绝不允许触发失效处理。未执行的源交易即使已经补偿, 仍保留其签名预留。

条件性进展。 固定一个 CAL 授权项。对于成员 m 和 Worker w , 令 $C_{m,w}$ 为本地观察到的切片总容量, 即 Available 与 Reserved 之和。令 $L_{m,w}$ 为指定请求集合之外的残留预留上界, 令 $N_{m,w} \geq 1$ 为该集合中可能同时取得或保留预留的不同请求数量上界, 其中包括候选请求。如果每个请求至多预留 $a_{\max}$ CAL, 且计入找零, 那么

$$C_{m,w} \geq L_{m,w} + N_{m,w} a_{\max}$$

可排除该候选请求因 CAL 容量不足而被拒绝的情形: 其他预留至多占用 $L_{m,w} + (N_{m,w} - 1)a_{\max}$ 。所有竞争预留都必须恰好计入一次, 包括部分批准或被放弃的批准、已认证但尚未释放预留的付款, 以及等待本地对账

的付款。

该条件必须在一个能够响应的诚实法定人数集合所涉及的相关扣减步骤和切片上成立。它是工作负载条件，而不是在多个收集者之间强制执行的上界。输入与 Intent 的兼容性、充足的费用及其他资源、可用证据、公平的消息交付与入块，仍是独立要求。源交易执行、经济闭合和表示各自具有不同的进展条件；任何一项都不蕴含普遍适用的完成期限或基于超时的解锁。

引理 12 (经济闭合不等待适配): 当公开截止期已到且决定被纳入区块时，尚未闭合的直接义务可以通过其公开补偿决定关闭，前提是该位置的原始历史、签发者储备和消费方费用托管均可用。该转移不需要任何适配贡献。表示的完成则另行要求有效适配材料、当前规范修订版本，以及公开提交和本地物理安装取得进展。

证明: 决定路径验证义务和历史槽位，并在不使用适配端点的情况下执行记账。由定理 1，储备充足；准入规则则在消费方的费用托管中，为每个认证输入预留一次补偿所需的费用。因此，适配不可用既不会阻止决定，也不会阻止后续偿还。门限服务需要四名固定持有者中的三名提供有效贡献；三名能够响应的诚实持有者已经足够，拜占庭持有者也可能提供有效贡献。共享执行资源和公开排序仍然影响时延，因此该结论区分的是协议依赖关系，而不给出完成时间上界。 ■

可用性与担保方风险。一旦消费方公开源交易 QC，至少两名诚实的源交易签署者就保有其提交材料（引理 4）。在公平的消息交付和入块条件下，这支持对可执行源交易的重提交，但不会消除永久性 Intent 冲突。到期义务可以转而在不依赖适配的情况下闭合（引理 12）。

在每个前缀，

$$0 \leq S_g = \sum_{c \in C_g} (p_c - \text{Rec}_c) \leq Q_g \leq B_g.$$

对于不再补资的固定授权项，该式限制的是未偿还补偿额，而不是整个生命周期内的累计补偿总额 Paid，也不代表各签名切片的可用性。即使仍有储备本金，认证也可能停止。当定理 3 的假设成立时，最终 CAL 持有情况和储备相同；永久未执行的源交易不属于该比较范围。因此，开放部署接受的是对抗性储备损失和容量持续占用，而不只是偶发源交易失败的某个预期比例。FUEL 费用并不为这些损失提供保险，本文也不假设任何 CAL-FUEL 兑换价值关系。

VI. 实现与评估

我们在冻结的评估构建 721800c 上评估已认证的继续、普通流量期间的来源解决以及本地历史读取。该构建集成了独立的赔付决定、公开证书来源恢复以及一对一的输入证书检查。该构建上的继续实验与混合负载实验使用相同的节点二进制与 Comet 源码；新增内容仅限于观测与离线测试工具。工件保留源码/overlay/二进制指纹以及逐次运行的记录。本地作废转换是在补丁 bf71c4d 中引入的。边界/重放回归以及注入延迟研究使用生产快照 9879f64。其测试与驱动程序的新增内容单独做了指纹；节点路径未变。这些功能性结果并不取代 721800c 的性能测量。

设置与测量。实验使用一台 Mac Studio M4 Max，配备 16 个 CPU 核心、64 GiB 内存、macOS 26.6.2 和 Go 1.27.1。CometBFT v0.38.26 的项目 overlay 实现身份与安装规则。网络试验在签名之前使用同步的 bbolt 成员提交；委员会应用状态、BlockStore 和网关均基于磁盘。成员使用 GOMAXPROCS=16 和 GOGC=200；提交、刷新和 gossip 的设置分别为 250、10 和 10 ms。网络试验复制相同的未运行创世与密钥，重置创世时间，并使用充足的固定资金而不充值。

已认证接收从第一次实际发送到接收方验证并原子地本地记录。公开与成员的观测包含区块跟随与驱动轮询；链实验另行记录应用提交的完成。除非另有说明，汇总取三次运行级统计量的中位数，而非合并后的交易分位数；范围描述运行之间的差异，而非置信区间。这些计时试验不测量重启恢复、可持续吞吐量或最低资本。下文的持久化重启检查是单独的功能性回归。

A. 无需逐跳结算的连续花费

九个服务支持三个独立的 NoSync 钱包沿 $A \rightarrow B \rightarrow C \rightarrow A$ 轮转 100 CAL，每笔支付使用所有者的最终 FUEL。在统一的三秒预热之后，后继交易根据前一接收方所存储的输出在线构造。快速模式在已认证接收之后继续；等待模式额外等待驱动程序对经钱包验证的公开执行的观察。三对运行按 fast/wait、wait/fast、fast/wait 的顺序进行。

两种模式都测量 $\text{Sent}_1 \rightarrow \text{Ready}_{100}$ ，包括第 2–100 跳的构造与重试，但不含第一笔支付的构造。等待模式增加 99 次跳间等待，不含最后一笔支付的公开确认。链时间的中位数为 4.557 s 对 64.476 s，即 14.15 倍之比和 92.93% 的缩减（图 4）；范围分别为 4.498–4.740 与

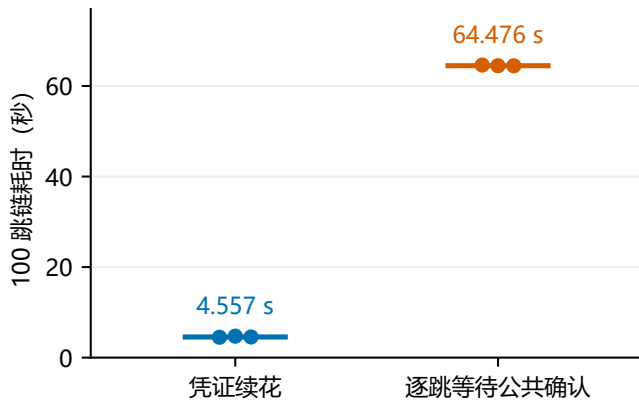


图 4. 构建 721800c 的 100 跳链：每种模式三次运行，成员同步、钱包 NoSync。两种模式都终止于最后一次已认证接收；只有等待模式要求在跳间进行公开观察。

64.461–64.627 s。快速模式用 300 次尝试完成 300 笔支付；等待模式使用 407 次尝试。该比较包含重试成本，但不把每一次重试或全部差异都归因于共识等待。

全部 297 个快速后继交易都使用其父交易的 TXCer，在父交易接收之后构造，并在四个副本中记录到的最早父交易应用提交完成之前首次发送。最小领先量为 224.785 ms。这些记录标记的是数据库提交的时间，而非共识决定的时间。之后的驱动观测并不是原始的钱包验证时间戳。实际的缺失输入记录（而非发送顺序）确立了 3、2 和 4 次子交易先于来源交易执行，且全部在无赔付的情况下被履行。

驱动程序每 5 ms 轮询一次，跟随者每 25 ms 轮询一次；250 ms 是提交设置。在每次等待运行的 99 次跳间等待中，从最早副本应用提交到驱动观测的总计为 29.542、29.681 和 29.347 s。该联合区间包含后继区块头的可用性、获取、验证、本地记录以及副本差异。钱包内部的时间戳不能区分这些项。因此，14.15 是所测得的端到端模式之比，而非纯粹的共识加速或轮询惩罚。补充材料给出了匹配的区间以及从观测到下一次构造的时间。

快速模式的中位数为：最后一笔支付的公开观测 5.228 s，所有支付的公开观测 5.229 s，所有支付的成员闭合 5.294 s。全部 600 笔支付都通过输入/输出、副本一致、CAL/FUEL、费用闭合和发件箱审计。每条链花费 9,400 FUEL：8,400 为奖励，1,000 被销毁。对于这一固定形态，TXCer 大小为 779 B；请求最初为 1,979 B，此后为 2,766 B，没有随祖先增长的情形。

B. 混合负载下的来源解决

四个委员会进程与两个组织（每个组织有四名成员和一个网关）构成 14 个服务。费用使用明确授权的由组织支付的 FUEL。普通钱包为 NoSync；场景钱包同步提交，并在每种处理中跟随区块至少 55 s。每次运行提供 7,000 笔独立的普通支付，目标为 100 笔/s 持续 70 s，限制为 64 个快速请求与 256 笔未完成支付。从驱动程序启动后十秒开始，八个独立的跨组织父-子对以一秒间隔依次启动，所用输入与 Intent 与普通流相分离。每种处理都包含这些对。

N 正常交付。R 扣留每个已认证父交易的发布者提交与 INSTALL，并保持公开证据恢复启用。C 使用相同的扣留，但禁用恢复。P 另外暂停所有适配端点，但不暂停共识或决定。C/P 在观察到各自的赔付之后释放每个迟到的父交易，不等待另一个目标或表示。P 仅在全四个副本报告全部八个目标均为 Recovered、且表示既未提交也未物化之后，才恢复适配。每种处理有三次全新运行，顺序为 N/R/C/P、P/C/R/N、N/R/C/P。

适配不可用期间的经济闭合。R 自动履行全部 24 个被扣留的来源，无需发布者重新提交或赔付。在这些目标上，来源执行在子交易发送之后 2.054–3.357 s 被观察到（中位数 2.905 s）。C/P 对 48 个目标各赔付 100 CAL，并回收每笔扣减：4,800 CAL 被赔付并被偿还。在这 48 个目标上，赔付观测出现在子交易发送之后 31.806–34.499 s，其中包括 30 秒的公开时间截止期、调度、执行和轮询。

在 P 中，全部 96 次副本-目标检查都在适配恢复之前确认了偿还，时间为第一笔普通发送之后 51.256–51.792 s。随后的每个表示与物化都在最后一笔普通发送之前完成。这一顺序在所测的执行中展示了适配不可用时的经济闭合（图 5），而不是由最终余额相符来推断其独立性。

共享资源代价与审计。表 II 同时报告被接纳的接收延迟与实际发送跨度及排空。所有 N/C 运行和两次 P 运行都达到 256 个未完成的上限；两次 C 运行和两次 P 运行还达到 64 个快速请求。从计划发送到实际发送所测得的调度滞后，其运行级 P95 达到 895.88 ms。P 第 1 次运行在 70 s 之后的尾部含 89 笔支付，接收 P95 为 282.61 ms；它不是完整的十秒窗口。因此较低的汇总分位数并不能确立性能改进或零干扰。所有运行都在最后一笔普通发送之后 0.815–1.356 s 排空。

十二次运行中的全部 84,192 笔支付都闭合。每次

表 II

混合负载下的普通支付测量。每种处理有三次运行，每次含 7,000 笔普通支付外加八个父-子对。表项为运行级统计量的中位数，最后一列除外，该列为各次运行中最大的调度滞后 P95。发送跨度与排空自实际发送起测量；排空是一项观测。

处理	接收 P50 (ms)	接收 P95 (ms)	接收 P99 (ms)	发送跨度 (s)	发送后排空 (s)	最大调度 P95 (ms)
N	55.13	152.77	243.87	70.245	1.015	255.40
R	53.03	139.89	220.06	69.990	0.890	0.24
C	54.01	136.49	227.58	70.395	1.029	700.33
P	55.00	144.21	227.52	70.576	1.155	895.88

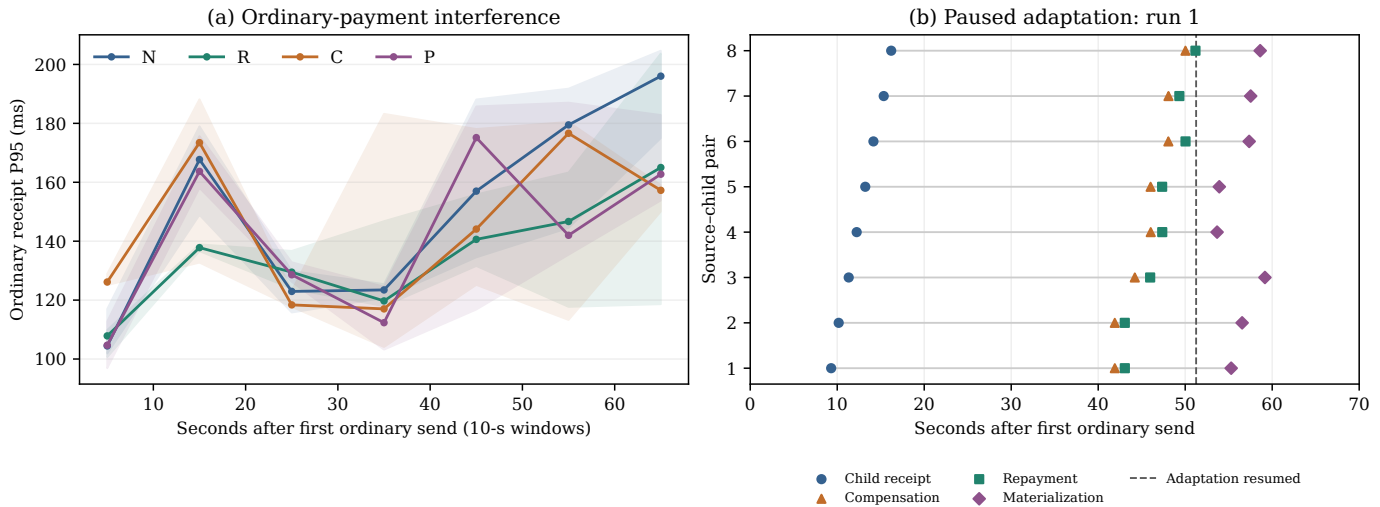


图 5. 混合负载观测。左：前七个十秒窗口中的普通接收 P95；线为三次运行的中位数，带为最小值-最大值，而非置信区间。右：P 第 1 次运行中的八个目标，展示适配恢复之前的赔付与偿还以及随后的物化。公开事件是观测，而非内部提交时间戳。

运行的四个副本在同一高度一致；CAL/FUEL 守恒、费用、身份、义务和发件箱均通过审计。各处理之间最终的用户 CAL 分配一致，备付金得以恢复，CAL/work 预留清空且无预算拒绝。N/R 每次运行花费 659,504 FUEL；C/P 花费 659,544，其中包括八笔额外的 5-FUEL 赔付费用。实际花费的费用不会被恢复为可用资金。

在聚合之前逐笔计算的归档计划到接收 P95，对 N/R/C/P 分别为 304/140/475/667 ms。这包括发送前的背压；补充材料将时间戳约定以及少量墙钟异常与实际发送接收延迟分开记录。

C. 历史读取：等价的答案与明确的代价

读取者是已验证并执行前缀 H 的诚实本地副本，在一个应用读事务 V_H 内查询原始坐标 (h, j, k) 。A 加载原始历史并使用永久决定索引；B 使用既有的 Canonical 修订。二者查阅相同的永久决定与义务，在 H 处返回相同的输入身份、TxID、坐标、金额、历史资金/扣减、决定/高度以及义务状态。授权来自绑定确切修订的成

功执行，而不是 CH 等式或匹配的 BlockID。这不是远程服务器认证。

测试夹具使用真实签名、成功的 ABCI 执行、基于磁盘的 bbolt/GoLevelDB 以及冻结的 BlockStore。A 拥有自己的原始 BlockStore，不含 B 的备份布局代价。点查询加载一个区块并解码所选交易；A 的区块查询扫描其范围索引一次并构建一个计时的映射。两种策略都不会获得预先计算的决策映射。页面是热的，没有已解码区块或结果缓存；GOMAXPROCS=16，GOGC=100。

三个全新进程各测试四个 (n, m) 夹具，其中 $n \in \{32, 256\}$ 为支付数， $m \in \{1, 8\}$ 为每个区块的已赔付槽位数。每种策略 20 次预热之后，对已赔付点、普通点和整区块各做每种策略 300 次交错查询，共得到 21,600 次计时查询。计时包含读事务边界、加载/解码、记录关联和结果构造。所有计时读取都使用已表示且已物化的 $H = 4$ ；其他阶段以及随后的真实偿还属于功能等价性检查，而非额外的计时样本。

Canonical 使已赔付点的 P50 降低 7.44–12.32%

表 III

历史读取与表示的代价。A 为原始历史加决定索引；B 为 CANONICAL。点读取以已赔付槽位为目标。查询表项为三个运行级 P50 的中位数；代价表项为三个总量的中位数。额外 KV 不含公共历史、决定和应用提交元数据。

(n, m)	点读取 (μs)		整区块 (μs)		额外净 KV (MiB/区块)	本地工作 (ms/区块)
	A	B	A	B		
(32, 1)	162.54	148.38	1,028.29	1,005.79	0.279	43.32
(32, 8)	169.21	156.62	1,127.46	1,083.71	0.314	94.46
(256, 1)	1,005.21	881.38	8,441.58	8,316.00	2.153	65.32
(256, 8)	1,006.25	887.25	8,554.17	8,407.12	2.189	214.40

(12.6–123.8 μs)；整区块的差异为 1.49–3.88%（表 III）。在 (256, 8) 第 1 次运行中，A 执行 12 次 BlockStore Get，返回 622,183 B；B 不执行任何 Get，但从应用状态读取 825,526 B。每次查询的平均堆分配约为 4.95 对 2.24 MB，而记录关联的中位数为 11.50 对 10.83 μs。这表明的是布局/重建方面的收益，而不是更少的总字节数或密码学加速。解码对象缓存可能改变这一权衡。

表示为每个被修复的区块增加 0.28–2.19 MiB 的净逻辑 KV，用于修订、任务以及备份/历史记录，已扣除被删除的 Pending。(256, 8) 的 5,056-B 批次已包含在内。本地顺序适配、组装、Finalize、同步的应用 Commit 与物化合计 43–214 ms。它不含网络/共识等待、公开区块保存以及提交签名的构造。公共的赔付成本另行报告。

等价性检查涵盖：先赔付后表示、先表示后物化，以及偿还时保留历史扣减且状态为 Recovered。无效坐标、跨前缀决定以及错误的扣减/金额都被拒绝；无法解码的修订会失败而不回退。这些检查并不确立对任意可解码的损坏的检测。Canonical 在稳定的历史坐标处提供授权的资金表示，并带有所测得的维护代价，而不是一个经济上必需的、或普遍更快的索引历史替代方案。

D. 注入延迟与单故障运行

在 9879f64 上，四种条件各运行三次，顺序交替。H 使用注入代理但不增加延迟；L 对成员 RPC 与委员会 P2P 流量每方向增加 25 ms。L-M 与 L-C 增加该延迟，并分别在 15 至 45 s 暂停成员 3 或委员会验证者 3。每次运行发送 100 笔预热支付、1,200 笔 20/s 的普通支付，以及两条在故障窗口内启动的 10 跳链。实际的健康检查 RPC RTT 从 1.17 升至 52.20 ms；RPC 与 P2P 的平均保持时间均约为每方向 25.2 ms。补充材料规定了 FIFO 注入、固定资源和逐次运行的结果。

表 IV

注入延迟与单故障：每次运行全部 1,200 笔普通支付的三个运行级统计量的中位数；预热与链不计入。公开与成员列为发送到观测的 P95；成员列等待全部四名成员。

条件	接收 P50/P95 (ms)	公开 (s)	成员 (s)
H	41.56/83.20	1.28	1.36
L	81.91/118.22	1.32	1.41
L-M	77.71/112.85	1.30	29.00
L-C	77.41/109.32	10.02	10.04

全部 15,840 笔支付均闭合，委员会状态相等，发件箱为空，公开缺口为零。无需赔付。表 IV 区分了接收、公开观测以及对全部四名成员的观测。暂停一名成员时，新的 QC 使用其余三名成员，公开完成继续进行，而缺席成员的观测造成 630–641 个未完成任务的峰值。暂停一名验证者时，接收 P95 保持在 108–111 ms，每个故障窗口内有 400–510 笔支付达到公开观测，但公开 P95 升至 9.5–12.6 s。保留的提交证书验证了每次运行中暂停验证者的五次第 0 轮提议者机会；每次都在第 1 轮提交，且不含该验证者的签名。两条链均在各自故障窗口内完成认证，且每次运行都在其最后一笔普通发送之后 1.25 s 内排空。这些是单机注入延迟的结果，与 fast/wait 对比相互独立。

E. 当前构建的边界与组合检查

9879f64 上的确定性测试使用实际的成员批准、ABCI 执行和经认证的跟随者更新。它们复现了无 QC 的额度碎片化、同输入 2/2 分裂，以及带完整 CAL/FUEL 核算的两轮 Intent 冲突赔付。外部充值解决了第一种情形，而不能解决第二种。真实的门限适配涵盖两种表示/偿还顺序；保留状态的重启复现每个原始命令历史。提交后、签名返回之前的中断保留锁与预留，并允许幂

等重试。补充材料给出了逐前缀的账本、负向入口测试和重启断言。

归档证据。补充材料保留了较早的继续、吞吐量、资本、组织路径和可用性实验，其构建与配置均保持原样。它们提供的是历史背景，而不是对 721800c 的测量；跨版本的差异不归因于个别改动。

VII. 结论

Next 分离了连续支付原本会串行化的三条路径。一轮认证使接收方能够花费新认证的输出，因此在成员同步提交的单台主机上，100 跳链到达其最后一次已认证接收只需 4.557 s，而非 64.476 s。直接责任消除了对来源的等待：由备付金支持的签名额度覆盖每份证书的未偿 CAL 责任，原签名者可根据已公布的证书重新提交被扣留的来源，而在所述协议条件下，公开决定可以关闭到期的 Open 义务，迟到的来源交易则将其偿还。在混合负载中，赔付与偿还在所有适配服务暂停期间完成。可选的授权表示在原始坐标处记录备付扣减，同时保持后继引用和完整 BlockID 不变，并伴随所测得的存储与维护代价。继续、经济闭合与历史记录各自基于其自身的证据推进。

致谢

ChatGPT 与 Claude 协助起草和修订摘要、正文、补充说明及中文阅读版。Codex 协助基于源码的审阅、实现与测试脚手架、修订、LaTeX 组装和绘图脚本。实验图表根据已记录的测量生成。

参考文献

- [1] M. Baudet, G. Danezis, and A. Sonnino, “FastPay: High-Performance Byzantine Fault Tolerant Settlement,” in *Proceedings of the ACM Conference on Advances in Financial Technologies*, 2020, pp. 163–177.
- [2] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, “Zef: Low-latency, Scalable, Private Payments,” in *Proceedings of the Workshop on Privacy in the Electronic Society*, 2023, pp. 1–16.
- [3] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, “Sui Lutris: A Blockchain Combining Broadcast and Consensus,” in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security*, 2024, pp. 2606–2620.
- [4] L. Aumayr, P. Moreno-Sanchez, A. Kate, and M. Maffei, “Blitz: Secure Multi-Hop Payments Without Two-Phase Commits,” in *30th USENIX Security Symposium*, 2021, pp. 4043–4060. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/aumayr>
- [5] —, “Breaking and Fixing Virtual Channels: Domino Attack and Donner,” in *Network and Distributed System Security Symposium*, 2023.
- [6] S. Sridhar, A. Sonnino, and L. Kokoris-Kogias, “Stingray: Fast Concurrent Transactions Without Consensus,” 2025, arXiv:2501.06531, version 1. [Online]. Available: <https://arxiv.org/abs/2501.06531>
- [7] S. Mazumdar and S. Ruj, “CryptoMaze: Privacy-Preserving Splitting of Off-Chain Payments,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 2, pp. 1060–1073, 2023.
- [8] V. Mavroudis, K. Wüst, A. Dhar, K. Kostianen, and S. Čapkun, “Snappy: Fast On-Chain Payments with Practical Collaterals,” in *Network and Distributed System Security Symposium*, 2020. [Online]. Available: <https://arxiv.org/abs/2001.01278>
- [9] A. Madhusudan, M. Sedaghat, S. Tiwari, K. Cong, and B. Preneel, “Reusable, instant and private payment guarantees for cryptocurrencies,” in *Information Security and Privacy (ACISP)*, 2023, pp. 580–605.
- [10] Z. Liu, Y. Xiang, J. Shi, P. Gao, H. Wang, X. Xiao, B. Wen, Q. Li, and Y.-C. Hu, “Make Web3.0 Connected,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 2965–2981, 2022.
- [11] G. Ateniese, B. Magri, D. Venturi, and E. Andrade, “Redactable Blockchain—or—Rewriting History in Bitcoin and Friends,” in *IEEE European Symposium on Security and Privacy*, 2017, pp. 111–126. [Online]. Available: <https://eprint.iacr.org/2016/757>
- [12] D. Zhang, J. Le, X. Lei, T. Xiang, and X. Liao, “Secure Redactable Blockchain With Dynamic Support,” *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 2, pp. 717–731, 2024.
- [13] X. Li, J. Xu, L. Yin, Y. Lu, Q. Tang, and Z. Zhang, “Escaping From Consensus: Instantly Redactable Blockchain Protocols in Permissionless Setting,” *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 5, pp. 3699–3715, 2023.
- [14] K. Y. Chan, L. Chen, Y. Tian, and T. H. Yuen, “Reconstructing Chameleon Hash: Full Security and the Multi-Party Setting,” in *Proceedings of the ACM Asia Conference on Computer and Communications Security*, 2024, pp. 1076–1091.
- [15] V. Shoup, “Practical Threshold Signatures,” in *Advances in Cryptology—EUROCRYPT*, 2000, pp. 207–220. [Online]. Available: <https://www.iacr.org/archive/eurocrypt2000/1807/18070209-new.pdf>
- [16] S. A. K. Thyagarajan, A. Bhat, B. Magri, D. Tschudi, and A. Kate, “Reparo: Publicly Verifiable Layer to Repair Blockchains,” in *Financial Cryptography and Data Security*, 2021, pp. 37–56. [Online]. Available: <https://arxiv.org/abs/2001.00486>
- [17] R. Matzutt, V. Ahlrichs, J. Pennekamp, R. Karwacik, and K. Wehrle, “A Moderation Framework for the Swift and Transparent Removal of Illicit Blockchain Content,” in *IEEE International Conference on Blockchain and Cryptocurrency*, 2022.
- [18] X. Chen and Y. Gao, “CDEdit: A Highly Applicable Redactable Blockchain with Controllable Editing Privilege and Diversified Editing Types,” 2022, arXiv:2205.07054. [Online]. Available: <https://arxiv.org/abs/2205.07054>
- [19] Cloudflare, “CIRCL v1.6.5: Threshold RSA implementation,” Software, tss/rsa package, accessed October 3, 2026. [Online]. Available: <https://github.com/cloudflare/circl/tree/v1.6.5/tss/rsa>
- [20] E. Buchman, J. Kwon, and Z. Milosevic, “The latest gossip on BFT consensus,” 2019, arXiv:1807.04938, version 3. [Online]. Available: <https://arxiv.org/abs/1807.04938v3>